Day 10 Revision Sheet

Oct 29, 2026 (finished Oct 30) | Excel: recall, Status column, references | Git: branches, merge, conflict, clone, pull | Smooth

1. Excel: recall, Status column, references

- **XLOOKUP vs VLOOKUP:** XLOOKUP takes a lookup range and a separate return range, so it can return from the left or the right, and it matches exactly by default. VLOOKUP counts a column number from the left edge, so it only returns to the right and breaks if a column is inserted. 4th XLOOKUP argument = text to show if not found.
- **Pivot source range:** Rows group, Values summarise. A new row outside the source range is NOT picked up, and Refresh alone will not fix that. Extend the source range, or build the pivot from an Excel Table (Ctrl+T), then Refresh. A pivot never updates itself.
- **#N/A from hidden characters:** compare `=LEN(E2)` with `=LEN(A2)` (different length = hidden characters), `=TRIM(A2)` for ordinary spaces, and `=SUBSTITUTE(A2,CHAR(160),"")` for the non-breaking space that TRIM misses. `EXACT` is case-sensitive, so it can point at a problem XLOOKUP does not have. `CHECK()` is not an Excel function. Function names are not case-sensitive.
- **Status column:** `=IF(AND(B2>1000,C2>100),"High","Low")`, plain comparisons on single cells. Boundary rows: O2 (Sales exactly 1000) and O3 (Profit exactly 100) stay Low. High rows: O1, O5, O6. `=COUNTIFS(B2:B9,">1000",C2:C9,">100")` = 3, same rows, because COUNTIFS pairs work as AND. With 'at least', O2 and O3 flip, so the count is 5; O4 (Sales 900) and O7 (Profit 99) are not ties and stay Low.
- **References revise:** `=B2*C2*(1+$E$1)`; row 4 (Bag) = 300 * 2 * 1.18 = 708. Without the `$`, D4 reads `=B4*C4*(1+E3)`; E3 is empty (0), so it silently shows 600 with no error. Offset from row 2 to row 4 is +2. `=COUNTIFS(B2:B6,">50",C2:C6,"<5")` = 2 (Bag, Shoe); Cap has Price 80 but Qty 5 is not below 5.

2. Git: branches, merge, conflict (took the longest)

2.1 Branch

- A branch is a **separate line of commits**, a movable label pointing at a commit. `main` is itself just one branch. Excel bridge: a copy of the workbook that remembers where it split from the original. Git does not copy files; it starts a new line of history, which is why branches are cheap.
- `git switch -c average-fix` creates the branch and moves onto it. A commit made there stays there: after `git switch main` the files in the folder swap back to main's version. `git branch` prints branch NAMES only, and `*` marks the one you are on (HEAD). `git log --oneline` shows only the current branch; add `--all --graph` to see every branch.

2.2 Merge

- `git merge average-fix` brings average-fix INTO the branch you are standing on. Merge combines the CHANGES; pasting a whole copy over the original would wipe other people's work.
- **Fast-forward:** main has no new commits since you branched, so Git only slides main's label forward. NO new commit is created (output: `Fast-forward`).
- **Diverged branches:** both have new commits, so a slide is impossible (main has a commit the other lacks). Git creates a **merge commit with 2 parents** (output: `Merge made by the 'ort' strategy`). If the two sides changed DIFFERENT files, or different lines, it combines them automatically with no conflict.

2.3 Conflict

- A conflict needs the **same lines of the same file changed differently on both sides**. Git pauses; `git status` says 'You have unmerged paths' and the file shows `UU`.
- The file now holds literal marker lines: `<<<<<<< HEAD` (your current branch), `=======`, `>>>>>>> average-fix` (the incoming branch). The program will not run, because the markers are invalid code: `SyntaxError`.

Step	What I do	Why
1	Edit the file: keep one side or combine; delete ALL 3 marker lines	Git never deletes the markers for me
2	`git add practice.py` (status now shows `M `)	tells Git this file is resolved and staged
3	`git commit` (message is pre-filled)	this commit finishes the merge; no second merge command
4	Run the program or tests	a file with no markers can still be wrong

- `git merge --abort` cancels a stopped merge. In Colab, if the file viewer throws a `setAttribute` JavaScript error, write the file from a cell with `%%writefile` or `%%bash` instead.

3. Git: clone, fetch, pull

- `git clone <URL>` makes a NEW folder with the files, the FULL history, `origin` already set, and the branch tracking origin. It needs no `git init` and no `git remote add`. Run into a non-empty folder it refuses: 'destination path already exists and is not an empty directory'. Cloning again never updates an old copy; it makes a second separate copy that lacks your unpushed commits.
- `git fetch` downloads new commits but does NOT touch your files or branch. `git status` then says 'behind by 1 commit', and `git log origin/main` lets you look first. `git pull` **= fetch + merge.** If you have no local commits it is a fast-forward; if you do, it can conflict exactly like section 2.3.
- `git remote add origin <url>` is only for a repo you started yourself with `git init`.

4. My mistakes and the fix

What I said / did	Correct version
'A branch is the main file or folder'	A separate line of commits. main is one branch among others.
Merge = 'copy and paste into the original'	Merge combines the changes. Pasting a whole copy can wipe other people's work.
`git branch` 'prints the commit message'	It prints branch names; * marks the current branch.
Merge 'creates a new commit'	Fast-forward only moves a label. Diverged branches get a merge commit with 2 parents.
No fast-forward 'because different files'	Because main has a commit the other branch lacks (diverged).
Conflict 'if the files are not there' / 'same folder'	Same lines of the SAME FILE changed on both sides.
Program fails 'because Git doesn't know what to do'	The marker lines are literal text in the file: SyntaxError.
Steps 'git add, commit, then merge'	Edit the file first. The commit completes the merge.
`git log branch/main`; 'clone again removes updates'	`git fetch` then `git log origin/main`. Clone makes a new separate copy, it does not update.
`EXACT()` and `CHECK()` for #N/A	LEN, TRIM, SUBSTITUTE(CHAR(160)). CHECK() does not exist.
'Refresh updates the pivot'	Extend the source range or use an Excel Table, then Refresh.
Pasted polished answers in a recall test	Recall must come from memory. Short and messy is useful; pasted is not.

5. How I improved today

- In the Status column I named the exact boundary for each row (1000 is not above 1000) and checked COUNTIFS against the IF results.
- I traced the fast-forward log correctly (new commit on top), resolved a real conflict in Colab, and counted the Excel offset exactly (+2).
- I said honestly that I had pasted some recall answers, which made the retest possible.

6. Practice to retry (no solutions)

1. Table Item, Price, Qty with a tax rate in F1: write a total formula that copies down. Then write the exact formula in row 5 if the $ were missing, and say what it calculates.
2. Extend the Status column to three levels: High if Sales above 1000 AND Profit above 100; Medium if only one of them is true; else Low. Predict all 8 rows first.
3. COUNTIFS for Price at least 50 and at most 300 (same column twice). Predict the count and name the boundary rows.
4. Make a branch, edit README.md on it and commit; edit practice.py on main and commit; merge. Predict: fast-forward or merge commit? Conflict or not? Why?
5. Create a conflict on the same line, resolve it by combining both ideas, then run the program. Say what `git status -s` shows at each stage (UU, then M).
6. After a fetch, status says 'behind by 2 commits'. Write the commands to inspect the new commits and then bring them in.
7. When do you use `git clone` and when `git init` plus `git remote add origin`? Give one real situation for each.

7. Recall questions (answer from memory)

1. Why does a missing $ on the tax rate give a silent wrong answer, not an error?
2. What is a branch, and what does the * in `git branch` mean?
3. When is a merge a fast-forward, and when does it create a merge commit with 2 parents?
4. What exactly causes a conflict, and what are the steps to fix one?
5. Why does the program fail while the markers are still in the file?
6. What is the difference between `git fetch` and `git pull`?
7. What does `git clone` give you that `git init` does not?

8. Open items

- Re-test: #N/A checks (LEN, TRIM, CHAR(160)), pivot source range, exact error names, origin vs main, fast-forward vs merge commit, conflict = same lines.
- `git push` from Colab was not retested (needs a token). Do it only with a throwaway repo.
- Commit the Day 9 and Day 10 README entries as separate commits with messages that say what changed.
- Oct 30 (Friday, Python day) tasks are next.